

Team 6 – Graded Lab – Synchronization mechanisms.

Jin Weijian Víctor, Álvaro Palancas Siguero, Javier Seguí Corbatón.

Exercise 1(Wizards_finalversion.py): Condition Variables.

1. Explain how you divided the problem into classes and threads. Why did you choose this division? What models each class?

We created 2 classes: Bridge and Wizard. Bridge contains the condition variable and the functions that control the access to the sections of the bridge. For each wizard an instance of the class Wizard is created (a thread). We chose this division because this way we can treat each wizard as its own thread, with its own values (direction and name).

2. What are the shared variables?

The shared variables are the bridge flow (direction), the number of wizards on the bridge, the number of wizards at Neville's side and the condition variable.

3. Identify the critical sections of your solution.

The critical sections of the code are all parts under the “with self.condition” indentation. That happens when a wizard enters or exits a section of the bridge, having to change the values of the shared variables.

4. What are the conditional variables used? What is protected by each conditional variable (if you use more than one)?

We only used the condition variable "Condition". All critical sections are protected by it, ensuring that the shared variables can only be changed by one thread at a time.

5. How do you ensure that the bridge only supports 5 wizards at a time?

We created a global variable called MAX_BRIDGE_CAPACITY, which defined the number of wizards allowed at the same time. We made all threads wait if there were already MAX_BRIDGE_CAPACITY wizards on the bridge using a “while” around a “self.condition.wait()”.

6. How do you ensure that the wizards can only cross in one direction?

Each wizard has its own direction so the “in_bridge” function receives the direction of the wizard that wants to cross and it checks that it is equal to the direction of the wizards who are in it or that the bridge is empty and the direction is none. If they cannot cross they stay in the “while” mentioned above.

7. How do you ensure that the square where the wizards are launching their spells only supports 10 wizards at a time?

We use a global variable called `MAX_NEVILLE_CAPACITY` and the conditional variable `nevilleside` to count the wizards that are launching their spells. When the wizards in the bridge plus the wizards in the Neville's side is equal to 10 no more wizards enter the bridge so the Neville's side cannot be more than 10. Again, if this condition is reached the wizard stays in the while mentioned above.

Exercise 2(Semaphores_finalversion.py): Semaphores.

1. Explain how you divided the problem into classes and threads. Why did you choose this division? What models each class?

We created 2 classes: `Battle` and `Wizard`. `Battle` contains the semaphores and the functions that control the access to the sections of the bridge. For each wizard an instance of the class `Wizard` is created (a thread). We chose this division because this way we can treat each wizard as its own thread, with its own values (direction and name).

2. What are the shared variables?

The shared variables are the bridge flow (direction), the number of wizards on the bridge, the number of wizards at Neville's side and the semaphores: `bridge`, `neville`, `ndqueue`, `hdqueue` and `mutex`.

3. Identify the critical sections of your solution.

The critical sections are all those parts between the acquisition and release of the mutex, which protects the shared variables.

4. What are the semaphores used? What is the initial value of each semaphore and why?

The semaphores used are: `bridge`, `neville`, `ndqueue`, `hdqueue` and `mutex`.

`Mutex` has an initial value of 1 because we can only allow 1 thread at a time to access the shared variables.

`Bridge` and `Neville` have an initial value of 5 and 10, because that's the amount of wizards that can be allowed in their respective section of the bridge.

`Ndqueue` has an initial value of `TOTALWIZARDS` because it must be able to hold all the wizards that have been initialized in the program.

`Hdqueue` has an initial value of `MAX_NEVILLE_CAPACITY` because it holds the wizards that are waiting to return to the bridge from Neville's side, which at most can be its max capacity.

5. How do you ensure that the bridge only supports 5 wizards at a time?

The bridge semaphore has capacity 5 so no more than 5 can get into the bridge. Before entering the bridge, a while checks if the max capacity of the bridge has been reached and if so, makes the wizard release the semaphores and wait again.

6. How do you ensure that the wizards can only cross in one direction?

When a wizard wants to enter the bridge its direction is compared to the flow of the bridge and if it is equal it can enter.

7. How do you ensure that the square where the wizards are launching their spells only supports 10 wizards at a time?

We have a variable that counts the wizards that are at the Neville's side and the wizards that are at the bridge. We ensure that no wizards go into the bridge the first time if the sum of the wizards on the bridge plus Neville's side is equal to or greater than 10.